Team Bio

Project: AdaptivePool: A Queue-Aware Adaptive Thread Pool for a Concurrent Web Server (OSTEP Project G.5)

Yang (Selena) Zhang

- **Email:** zhang.yang18@northeastern.edu
- **GitHub:** yangzhang98
- **Role (lead):** Workload generator and instrumentation. I will build the load generator that produces steady and bursty traffic with different CPU / I/O mixes, and the timestamp logging we use to measure latency.
- **Strengths for this project:** Backend development experience from my internship at a logistics company, where I built a shipment tracking and anomaly alerting platform that dealt with a steady stream of incoming events. I am comfortable with Python for scripting experiments and plotting results.
- **Project goal:** Produce clean, repeatable experiments that clearly show whether the adaptive pool helps tail latency.
- **Something I want to learn:** How to write correct multi-threaded C code with pthreads, mutexes and condition variables, and how to measure tail latency properly.

Minyuan Wang

- **Email:** wang.miny@northeastern.edu
- **GitHub:** minyuan1209
- **Role (lead):** Baseline concurrent server in C. Turn the single-threaded OSTEP `wserver` into a master/worker design: a fixed pool of pthreads, a bounded request buffer protected by one mutex and two condition variables (`not_empty`, `not_full`), the `-t`, `-b` and `-s` command-line options, FIFO and Smallest-File-First scheduling, and the `..` path check.
- **Strengths for this project:** Comfortable with C and low-level programming, including pointers, memory management, and building with `gcc` and `make` on Linux. Careful about edge cases, which matters for code that every other part of the project depends on.
- **Project goal:** Deliver a correct, stable baseline server early in Sprint 2 with no busy waiting, no deadlocks and no race conditions, so the team has a solid foundation to build the extension and run experiments on.

- **Something I want to learn:** How to design and debug producer/consumer synchronization with pthread mutexes and condition variables, and how to use tools like `gdb`, `valgrind` and ThreadSanitizer to find concurrency bugs.

Yao Qin

- **Email:** qin.yao1@northeastern.edu
- **GitHub:** Aureliasusu
- **Role (lead):** AdaptivePool research extension in C. Build the monitor thread that samples queue length and idle workers, the grow-fast / shrink-slow policy with hysteresis, and the safe retirement protocol (`retire_target` plus a broadcast on `not_empty`) so workers only exit between requests. Make min/max pool size, watermarks and sampling interval configurable from the command line.
- **Strengths for this project:** Good at reasoning about system design and trade-offs, and comfortable reading research papers and turning an idea into working code. Background in C and data structures from systems coursework.
- **Project goal:** Show whether a simple queue-aware controller can match the tail latency of a large fixed pool under bursty traffic while keeping fewer threads alive, and explain clearly when it helps and when it does not.
- **Something I want to learn:** How to safely create and retire threads at runtime without leaking resources or losing requests, and how feedback-control ideas from systems research like SEDA work in a real server.

Shared Responsibilities

- Sprint 1 work (exploration of the three existing systems, sequence diagrams, proposal) was done together by all three of us. The lead roles above start in Sprint 2.
- Everyone logs their time in the contribution tracker after each work session.
- Experiments and the final analysis (Sprint 3) are done together, with the workload owner coordinating the runs.